

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Tejaswi Reddy K	Reg. No.:	192371036
Section / Batch:	CS-BIOSCI	Date:	20-08-2026

Instructions to Students

- Answer both questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identifies all key issues, in-depth analysis, Innovative, feasible solutions with strong justification

Question Type	Description	Questions	Total Marks
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1 – Q2	100

SECTION A – CASE STUDY

Home - Classroom

SIMATS Online Lab

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1345

Search

80%

Star

Download

Print

School



SIMATS Online Programming Lab
OAA PROGRAMMING

KOMMURU TEJASWI REDDY
1923702440

QUESTIONS

Q1
Divide and Conquer
50 marks

Q2
Divide and Conquer
50 marks

2/2 answered

Q1 Divide and Conquer

50 marks

Case Study: Log File Analysis System (Binary Search + Sorting)
A system analyzes large log files by first sorting entries and then searching for specific events. Analyze how sorting (Merge/Quick Sort) and Binary Search can be combined effectively. Identify key issues such as processing time and data size. Apply divide and conquer techniques in both stages. Analyze overall time complexity. Design a solution and justify its efficiency.

Your Code

```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = merge_sort(arr[:mid])
6     right = merge_sort(arr[mid:])
7     return merge(left, right)
8
9 def merge(left, right):
10    result = []
11    i = j = 0
12    while i < len(left) and j < len(right):
13        if left[i] <= right[j]:
14            result.append(left[i])
15            i += 1
16        else:
17            result.append(right[j])
18            j += 1
19    result.extend(left[i:])
20    result.extend(right[j:])
21    return result
22
23 # Example usage:
24 logs = [5, 2, 8, 3, 7, 1]
25 logs_sorted = merge_sort(logs)
26 target = 3
27 idx = binary_search(logs_sorted, target)
28 print("Sorted logs:", logs_sorted)
29 print("Target found at index:", idx if idx != -1 else "Not found")
```

Input

stdin Input

Output

Sorted logs: [1, 2, 3, 5, 7, 8]
Target found at index: 4

Activate Windows
Go to Settings to activate Windows.

Home - Classroom

SIMATS Online Lab

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1345

Search

80%

Star

Download

Print

School



SIMATS Online Programming Lab
OAA PROGRAMMING

KOMMURU TEJASWI REDDY
1923702440

QUESTIONS

Q1
Divide and Conquer
50 marks

Q2
Divide and Conquer
50 marks

2/2 answered

Q1 Divide and Conquer

50 marks

Case Study: Log File Analysis System (Binary Search + Sorting)
A system analyzes large log files by first sorting entries and then searching for specific events. Analyze how sorting (Merge/Quick Sort) and Binary Search can be combined effectively. Identify key issues such as processing time and data size. Apply divide and conquer techniques in both stages. Analyze overall time complexity. Design a solution and justify its efficiency.

Your Code

```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = merge_sort(arr[:mid])
6     right = merge_sort(arr[mid:])
7     return merge(left, right)
8
9 def merge(left, right):
10    result = []
11    i = j = 0
12    while i < len(left) and j < len(right):
13        if left[i] <= right[j]:
14            result.append(left[i])
15            i += 1
16        else:
17            result.append(right[j])
18            j += 1
19    result.extend(left[i:])
20    result.extend(right[j:])
21    return result
22
23 # Example usage:
24 logs = [5, 2, 8, 3, 7, 1]
25 logs_sorted = merge_sort(logs)
26 target = 3
27 idx = binary_search(logs_sorted, target)
28 print("Sorted logs:", logs_sorted)
29 print("Target found at index:", idx if idx != -1 else "Not found")
```

Input

stdin Input

Output

Sorted logs: [1, 2, 3, 5, 7, 8]
Target found at index: 4

Activate Windows
Go to Settings to activate Windows.

Home - Classroom

SIMATS Online Lab

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1345

Search

80

Star

Download

Print

School

1



SIMATS Online Programming Lab
OAA PROGRAMMING

KOMMURI TEJASWI REDDY
1923702460

QUESTIONS

Q1
Divide and Conquer
30 marks

Q2
Divide and Conquer
30 marks

2/2 answered

Q1 Divide and Conquer 30 marks

30 marks

Case Study: Airline Reservation Sorting System (Merge Sort)
An airline reservation system sorts flight bookings based on departure time and priority. Analyze how Merge Sort ensures stable and efficient sorting. Identify issues such as large data volume and real-time updates. Apply divide and conquer techniques to explain the process. Analyze time complexity and design a solution, justifying its effectiveness.

Your Code

```
1 from dataclasses import dataclass
2
3 @dataclass
4 class Booking:
5     departure_time: int  # e.g., minutes since midnight
6     priority: int        # higher number = higher priority (e.g., 1000)
7     original_index: int  # to preserve stability
8
9 def merge(left, right):
10     merged = []
11     i = j = 0
12
13     # Merge process
14     # If left and right have equal elems, take from left first
15     while i < len(left) and j < len(right):
16         if left[i] <= right[j]:
17             merged.append(left[i])
18             i += 1
19         else:
20             merged.append(right[j])
21             j += 1
22
23     # Append remaining elements
24     merged.extend(left[i:])
25     merged.extend(right[j:])
26     return merged
27
28 def merge_sort(bookings):
29     if len(bookings) <= 1:
30         return bookings
31
32     mid = len(bookings) // 2
33     left = merge_sort(bookings[:mid])
34     right = merge_sort(bookings[mid:])
35     return merge(left, right)
36
37 # Sample usage
38 if __name__ == "__main__":
39     bookings = [
40         Booking(100, 2, 0),  # 10:00 priority 2
41         Booking(300, 1, 1),  # 3:00 priority 1
42         Booking(500, 3, 1),  # 5:00 priority 3
43         Booking(600, 3, 3),  # 6:00 priority 3 (same time -> stable sort)
44     ]
45
46     sorted_bookings = merge_sort(bookings)
47     for b in sorted_bookings:
48         print(b)
```

Input

stdin input

Output

```
Booking(departure_time=100,
priority=2, original_index=0)
Booking(departure_time=300,
priority=1, original_index=1)
Booking(departure_time=500,
priority=3, original_index=2)
Booking(departure_time=600,
priority=3, original_index=3)
```

Activate Windows
Go to Settings to activate Windows.

Home - Classroom

SIMATS Online Lab

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1345

Search

80

Star

Download

Print

School

1



SIMATS Online Programming Lab
OAA PROGRAMMING

KOMMURI TEJASWI REDDY
1923702460

QUESTIONS

Q1
Divide and Conquer
30 marks

Q2
Divide and Conquer
30 marks

2/2 answered

Q1 Divide and Conquer 30 marks

30 marks

Case Study: Airline Reservation Sorting System (Merge Sort)
An airline reservation system sorts flight bookings based on departure time and priority. Analyze how Merge Sort ensures stable and efficient sorting. Identify issues such as large data volume and real-time updates. Apply divide and conquer techniques to explain the process. Analyze time complexity and design a solution, justifying its effectiveness.

Your Code

```
1 from dataclasses import dataclass
2
3 @dataclass
4 class Booking:
5     departure_time: int  # e.g., minutes since midnight
6     priority: int        # higher number = higher priority (e.g., 1000)
7     original_index: int  # to preserve stability
8
9 def merge(left, right):
10     merged = []
11     i = j = 0
12
13     # Merge process
14     # If left and right have equal elems, take from left first
15     while i < len(left) and j < len(right):
16         if left[i] <= right[j]:
17             merged.append(left[i])
18             i += 1
19         else:
20             merged.append(right[j])
21             j += 1
22
23     # Append remaining elements
24     merged.extend(left[i:])
25     merged.extend(right[j:])
26     return merged
27
28 def merge_sort(bookings):
29     if len(bookings) <= 1:
30         return bookings
31
32     mid = len(bookings) // 2
33     left = merge_sort(bookings[:mid])
34     right = merge_sort(bookings[mid:])
35     return merge(left, right)
36
37 # Sample usage
38 if __name__ == "__main__":
39     bookings = [
40         Booking(100, 2, 0),  # 10:00 priority 2
41         Booking(300, 1, 1),  # 3:00 priority 1
42         Booking(500, 3, 1),  # 5:00 priority 3
43         Booking(600, 3, 3),  # 6:00 priority 3 (same time -> stable sort)
44     ]
45
46     sorted_bookings = merge_sort(bookings)
47     for b in sorted_bookings:
48         print(b)
```

Input

stdin input

Output

```
Booking(departure_time=100,
priority=2, original_index=0)
Booking(departure_time=300,
priority=1, original_index=1)
Booking(departure_time=500,
priority=3, original_index=2)
Booking(departure_time=600,
priority=3, original_index=3)
```

Activate Windows
Go to Settings to activate Windows.

Code of Conduct

I Tejaswi Reddy K(192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Tejaswi Reddy K

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Mark Evaluation:

Case Study	Understanding of case (25)	Applications of concepts (25)	Analysis (20)	Solution Design (20)	Clarity (10)	Total Marks (Each - 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge

Course Coordinator

Head of Department

Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____